Advanced Programming

Generics

The Context

- “Create a data structure that stores elements:
  - *a stack, a linked list, a vector*
  - *a graph, a tree, etc.”*
- What data type to use for representing the elements of the structure?

Homogenous structure	Heterogeneous structure

`class Stack {`
`  private int[] items;`
`  public void push (int item) { ...}`
`  public int peek() { … }`
`}`
`…`
`Stack stack = new Stack();`
`stack.push(100);`
`stack.push(200);`
~~`stack.push("Hello World!");`~~
	`class Stack {`
	`  private Object[] items;`
	`  public void push (Object item) {...}`
	`  public Object peek() { … }`
	`}`
	`…`
	`Stack stack = new Stack();`
	`stack.push(100);`
	`stack.push(new Rectangle());`
	`stack.push("Hello World!");`
	`String s = (String) stack.peek;`

Generics

- Generics enable types (classes and interfaces) to be parameters when defining classes, interfaces and methods.

```
Stack<String> stack = new Stack<String>();
```

- Stronger type checks at compile time.

```
stack.push(new Rectangle());
```

- Elimination of casts.

```
String s = (String) stack.peek();
```

- Enabling generic algorithms.

Defining a Generic Type

```
class ClassName<T1, T2, ..., Tn> { … }
```

or

```
interface IName<T1, T2, ..., Tn> { … }
```

```
/**
 * A generic version of the Stack class
 * @param <E> the type of the elements
 */
public class Stack<E> {

    // E is a generic data type
    private E[] items;

    public void push(E item) { ... }
    public E peek() { .. }
}
```

E is the type parameter

Type Parameter Naming Conventions

- E - Element (used extensively by the Java Collections Framework)
- K - Key
- N - Number
- T - Type
- V - Value
- S,U,V etc. - 2nd, 3rd, 4th types

```
public class Node<T> { … }
public interface Pair<K, V> { … }
public class PairImpl<K, V> implements Pair<K, V> {...}
```

Example: *Pair*

```
public class Pair<K, V> {

    private K key;
    private V value;

    public Pair(K first, V second) {
        this.key = first;
        this.value = second;
    }
    public void setKey(K key) {
        this.key = key;
    }
    public void setValue(V value) {
        this.value = value;
    }
    public K getKey() {
        return key;
    }
    public V getValue() {
        return value;
    }
}
```

An old debate:
Do we want a java.util.Pair ?

Instantiating a Generic Type

- Generic Invocation

String is the type argument

```
Stack<String> stack = new Stack<String>();

Pair<Integer,String> pair =

          new Pair<Integer,String>(0, "ab");
Stack<Node<Integer>> nodes = new Stack<Node<Integer>>();
```

- *The Diamond* **<>**

```
Stack<String> stack = new Stack<>();

Pair<Integer,String> pair = new Pair<>(0, "ab");

Stack<Node<Integer>> nodes = new Stack<>();
```

The compiler can determine, or infer, the type arguments from the context.

Generic Methods

Generic methods are methods that introduce their own type parameters.

```
public class Util {
  public static <T> int countNullValues(T[] anArray) {
    int count = 0;
    for (T e : anArray)
        if (e == null) {
            ++count;
        }
    return count;
  }
}
Util.countNullValues(new String[]{"a", null, "b"});
Util.countNullValues(new Integer[]{1, 2, null, 3, null});
```

Bounded Type Parameters

```
class D <T extends A & B & C> { /* ... */ }
```

```
public class Node<T extends Number> {
    private T t;
    public void set(T t) { this.t = t; }
    public T get() { return t; }

    // Generic Method
    public <U extends Integer> void inspect(U u){
        System.out.println("T: " + t.getClass().getName());
        System.out.println("U: " + u.getClass().getName());
    }

    public static void main(String[] args) {
        Node<Double> node = new Node<>();
        node.set(12.34);                //OK
        node.inspect(1234);             //OK
        node.inspect(12.34);            //compile error!
        node.inspect("some text");      //compile error!
    }
}
```

Generics, Inheritance, Subtypes

"is a"

- ✔ `Integer extends Object`
- ✔ `Integer extends Number`
- • ~~**`Stack<Integer> extends Stack<Object>`**~~
- • ~~**`Stack<Integer> extends Stack<Number>`**~~

Given two concrete types A and B (for example, *Number* and *Integer*), *MyClass<A>* has no relationship to *MyClass<B>*, regardless of whether or not A and B are related. The common parent of *MyClass<A>* and *MyClass<B>* is *Object*.

"This is a common misunderstanding when it comes to programming with generics, but it is an important concept to learn "

Variance

- **Variance** refers to how subtyping between more complex types relates to subtyping between their components.

```
class Animal  {}
class Cat extends Animal { }

Cat tom = new Cat();
Animal[] pets = new Animal[10];
Cat[] cats = new Cat[10];

tom instanceof Animal      → true
cats instanceof Animal[]   → true   (covariance)
pets instanceof Cat[]      → false  (contravariance)

List<Animal> pets = new ArrayList<>();
List<Cat> cats = new ArrayList<>();
cats instanceof List<Animal>   → compile error
                                 (illegal generic type)

void play(List<Animal> pets) { … }
play(cats); → compile error (incompatible types)
```

Wildcards

- Upper bounded

```
public double sumOfList(List<? extends Number> list) {
    //it works on List<Integer>, List<Double>, List<Number>, etc.
    double s = 0.0;
    for (Number n : list)
        s += n.doubleValue();
    return s;
}
```

- Unbounded

```
public void printList(List<?> list) {
    for (Object elem: list)
        System.out.print(elem + " ");
}
```

- Lower bounded

```
public void addNumbers(List<? super Integer> list) {
    //it works on List<Integer>, List<Number>, and List<Object> –
    //anything that can hold Integer values.
    for (int i = 1; i <= 10; i++) {
        list.add(i);
    }
}
```